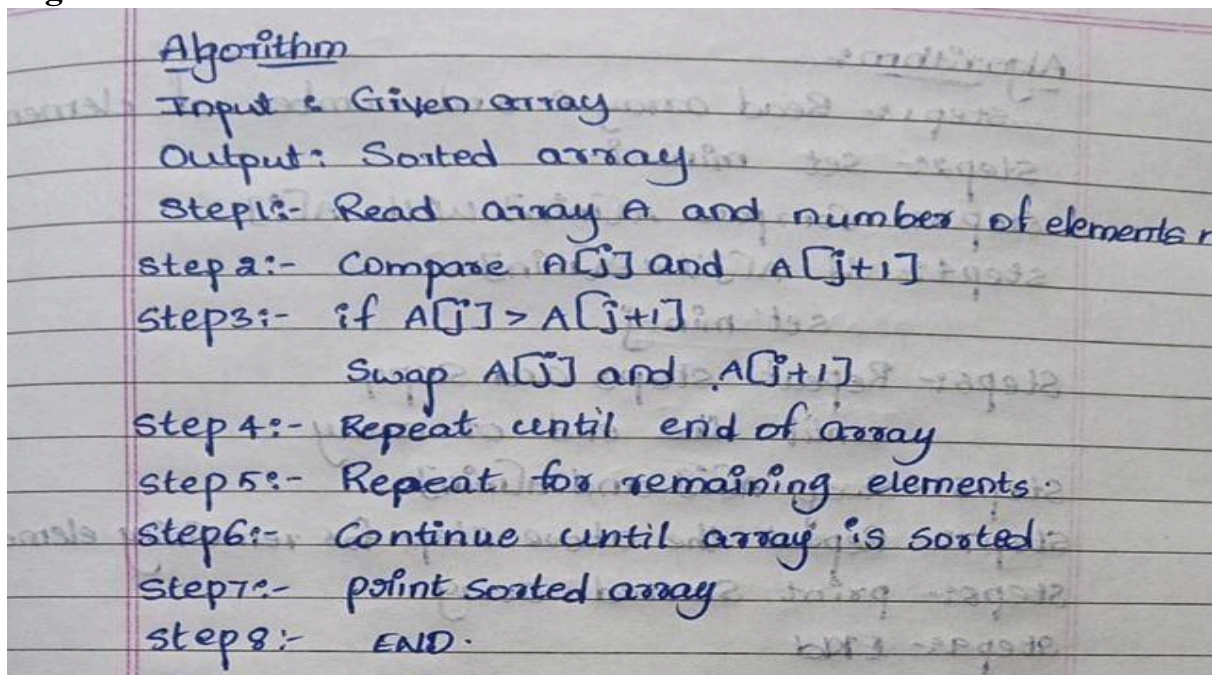


Practical 1

Aim: To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

1)Bubble sort :

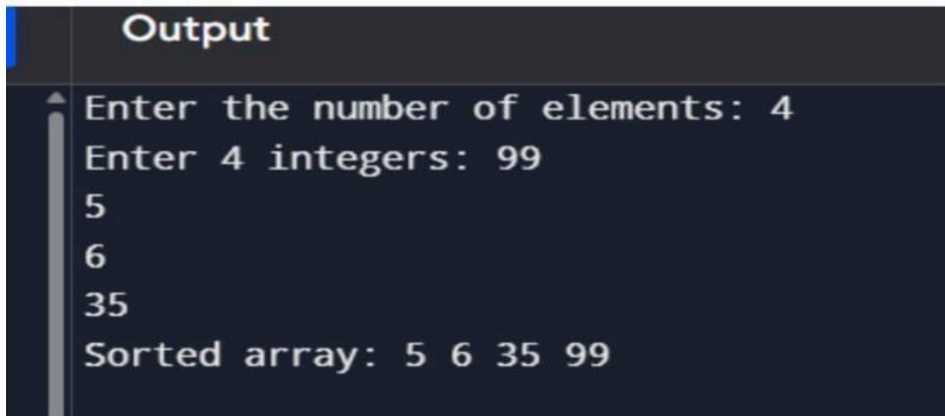
Algorithm:



Code for bubble sort:

```
#include <iostream>
#include <vector> using namespace
std;
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
    }  
}  
int main() {  
    int size;  
    cout << "Enter the number of elements:  
"; cin >> size; vector<int> arr(size);  
    cout << "Enter " << size << " integers:  
"; for (int i = 0; i < size; ++i) {    cin >>  
arr[i];  
    }  
    bubbleSort(arr);  
    cout << "Sorted array: ";  
    for (int i = 0; i < size; ++i) {  
    cout << arr[i] << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

Output:A screenshot of a terminal window with a dark background. The title bar says "Output". The text in the terminal shows the program's execution: it prompts for the number of elements (4), then prompts for 4 integers (99, 5, 6, 35), and finally displays the sorted array: 5 6 35 99.

```
Output  
Enter the number of elements: 4  
Enter 4 integers: 99  
5  
6  
35  
Sorted array: 5 6 35 99
```

Time complexity:

Page No. _____

Time Complexity:-

Best Case:-

The array is sorted

$n-1$ Comparisons

$T(n) = n-1$

$T(n) = O(n)$

Average Case:-

$(n-1) + (n-2) + (n-3) + \dots + 2 + 1$

$1 + 2 + \dots + (n-1)$

$T(n) = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}$

$T(n) = O(n^2)$

Worst Case:-

The array is in reverse order.

ex: 50 40 30 20 10

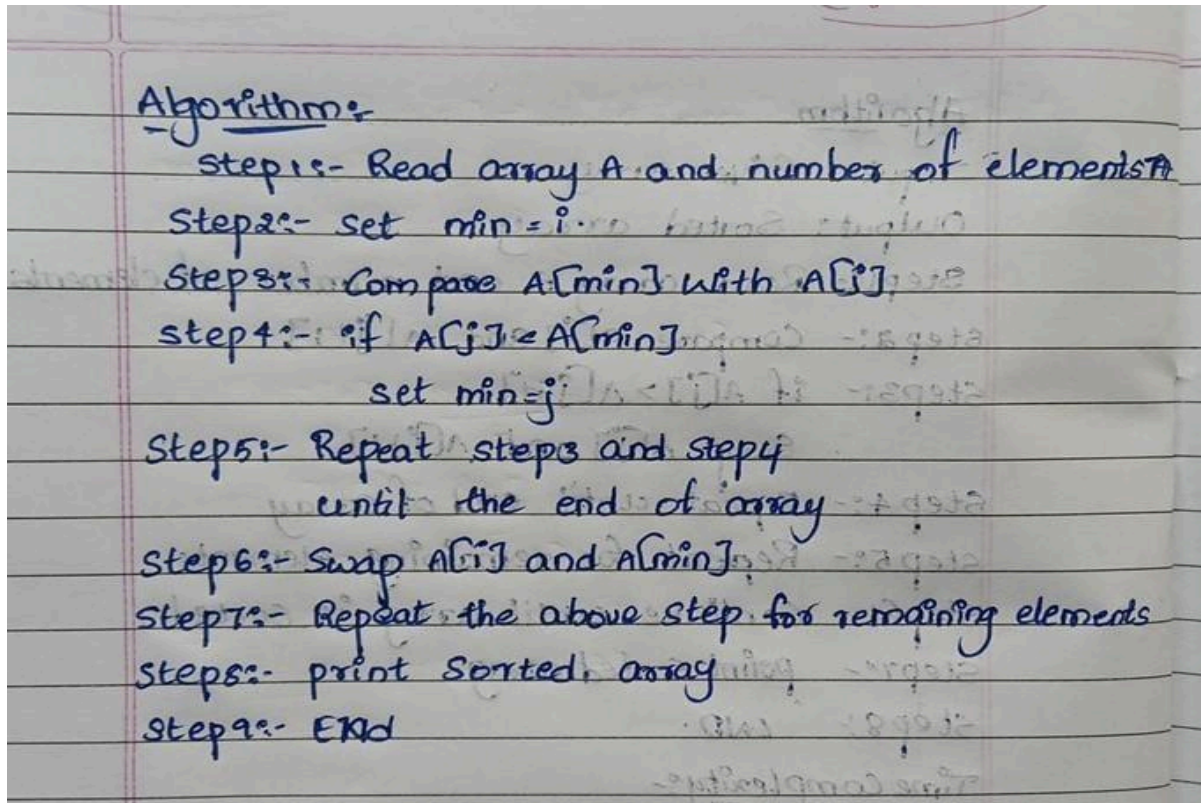
$(n-1) + (n-2) + \dots + 2 + 1$

$\frac{n(n-1)}{2} = \frac{n^2 - n}{2}$

$T(n) = O(n^2)$

2) Selection sort

Algorithm for selection sort:



Code for selection sort:

```
#include <iostream>
#include <vector>
void selectionSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        int minIndex = i;
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            std::swap(arr[i],
                arr[minIndex]);
        }
    }
}
```

```
int main() {
int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;

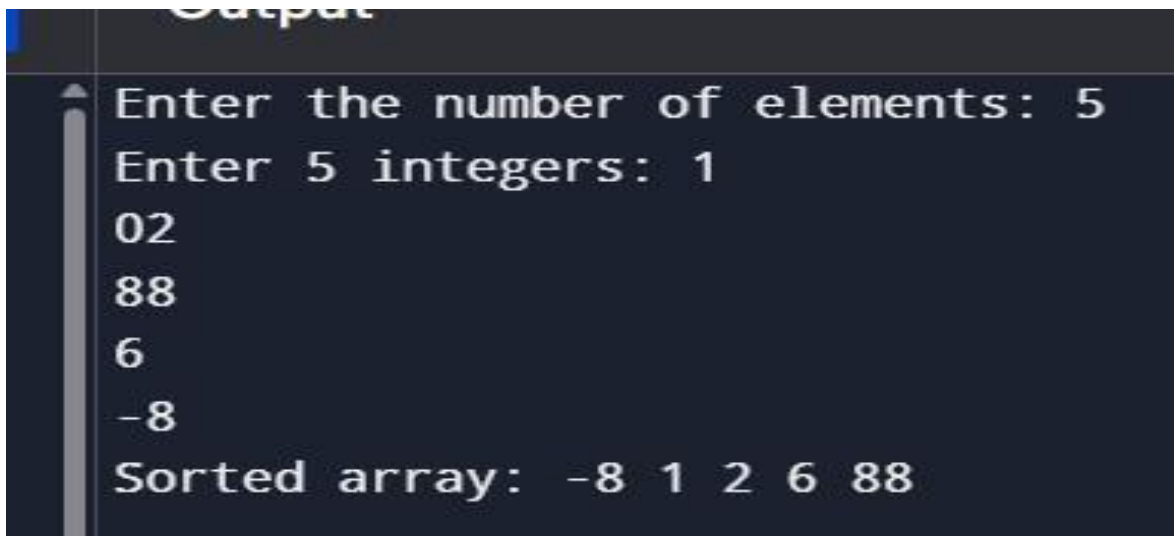
    if (n <= 0) {
return 0;
    }

    std::vector<int> arr(n);
    std::cout << "Enter " << n << " integers:
"; for (int i = 0; i < n; ++i) {    std::cin >>
arr[i];
    }

    selectionSort(arr);

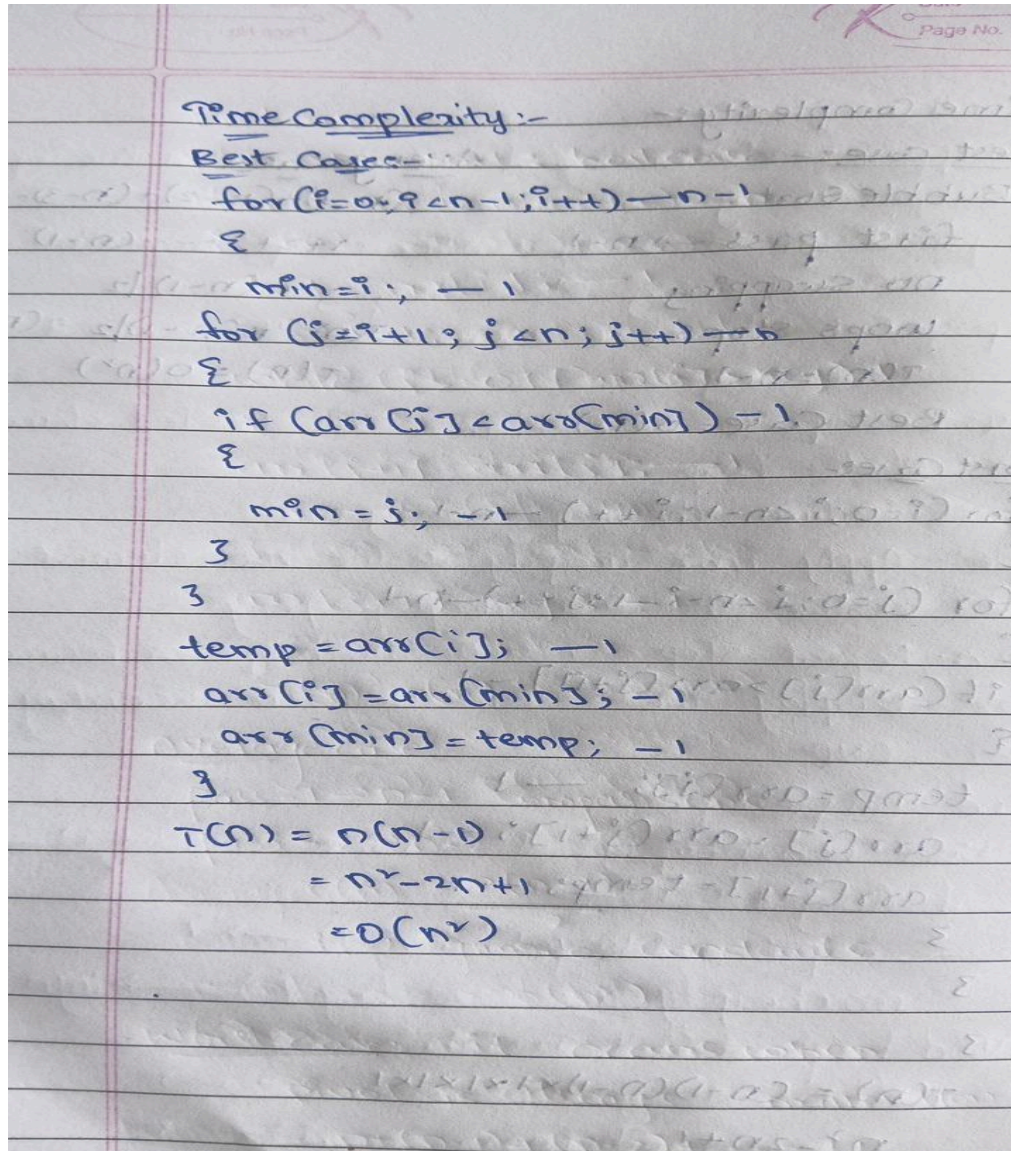
    std::cout << "Sorted array: ";
    for (int i = 0; i < n; ++i) {
std::cout << arr[i] << " ";
    }    std::cout <<
std::endl;    return 0;
}
```

Output:



```
Enter the number of elements: 5
Enter 5 integers: 1
02
88
6
-8
Sorted array: -8 1 2 6 88
```

Time complexity:



Average Case:-

for (i=0; i < n-1; i++) — n-1

{

min = i; — 1

for (j = i+1; j < n; j++) — n

{

if (arr[j] < arr[min]) — 1

{

min = j; — 1

}

}

temp = arr[i]; — 1

arr[i] = arr[min]; — 1

arr[min] = temp; — 1

}

$T(n) = n(n-1)$

$= n^2 - 2n + 1$

$= O(n^2)$

Worst Case:-

$T(n) = (n-1) + (n-2) + \dots + 1$

$= n(n-1)/2$

$T(n) = O(n^2)$

3) Insertion sort

Algorithm:

Algorithm:-

Inputs:- Give array

Output:- Sorted array

Step1:- Read array A and n

Step2:- Set key = A[i]

Step3:- Compare key with A[j]

Step4:- if A[j] > key,
Shift A[j] one position right

Step5:- Repeat until the correct position for key is found

Step6:- Insert key at its correct position

Step7:- Repeat for all remaining elements

Step8:- print sorted array

Step9:- END.

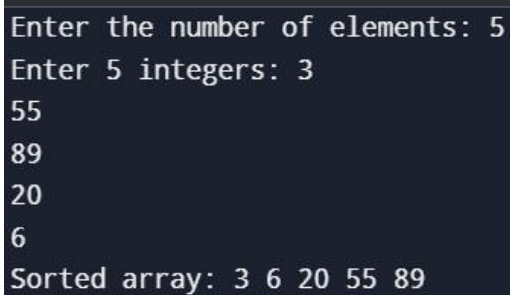
Code:

```
#include <iostream>
#include <vector> void
insertionSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;
```

```
if (n <= 0) {  
return 0;  
}  
std::vector<int> arr(n);  
std::cout << "Enter " << n << " integers: ";  
for (int i = 0; i < n; ++i) {  
std::cin >> arr[i];  
}  
  
insertionSort(arr);  
  
std::cout << "Sorted array: ";  
for (int i = 0; i < n; ++i) {  
std::cout << arr[i] << " ";  
} std::cout <<  
std::endl; return 0;  
}
```

Output:



```
Enter the number of elements: 5  
Enter 5 integers: 3  
55  
89  
20  
6  
Sorted array: 3 6 20 55 89
```

Time complexity:

Best Case:-

Insertion sort: best case

for ($i=1; i < n; i++$) — n

{

key = arr[i]; — 1

j = i - 1; — 1

while ($j >= 0$ & arr[j] > key) — 1

{

arr[j+1] = arr[j]; — 1

j--;

}

arr[j+1] = key; — 1

}

$T(n) = O(n)$

Average Case:-

pass-1 $\rightarrow n-1$ Comparisons

pass-2 $\rightarrow n-2$

$$T(n) = \frac{n(n-1)}{2}$$

ignore constants

$$= O(n^2)$$

Worst Case:- it occurs when

pivot is always smaller than largest

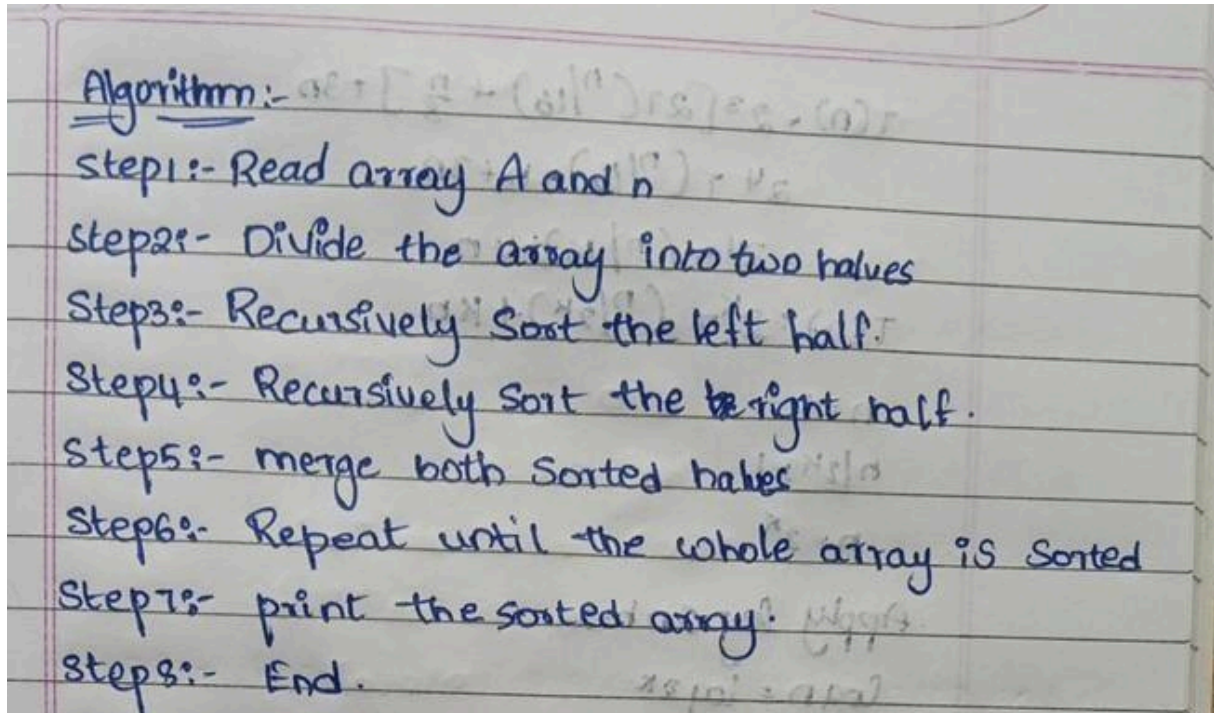
$$n + (n-1) + (n-2) + \dots + 1$$

$$T(n) = \frac{n(n+1)}{2}$$

$$= O(n^2)$$

Merge Sort:

Algorithm:



Code:

```
#include <iostream>
#include <vector>
void merge(std::vector<int>& arr, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - left;
    std::vector<int> L(n1);
    std::vector<int> R(n2);
    for (int i = 0; i < n1; ++i) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; ++j) {
        R[j] = arr[mid + 1 + j];
    }
    int i = 0;
    int j = 0;
    int k = left;
    while (i < n1
    && j < n2) {
        if (L[i]
        <= R[j]) {
            arr[k] =
            L[i];
            ++i;
        }
        else {
            arr[k]
            = R[j]; ++j;
        }
    }
}
```

```
        }
        ++k;
    }
    while (i < n1) {
arr[k] = L[i];
        ++i;
        ++k;
    } while (j <
n2) { arr[k]
= R[j];
        ++j;
        ++k;
    }
}

void mergeSort(std::vector<int>& arr, int left, int right) {
if (left < right) {
    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}
}

int main() {
int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;

    if (n <= 0) {
return 0;
    }

    std::vector<int> arr(n);    std::cout <<
"Enter " << n << " integers: ";    for (int i
= 0; i < n; ++i) {    std::cin >> arr[i];
    }    mergeSort(arr, 0, n - 1);
    std::cout << "Sorted array: ";
    for (int i = 0; i < n; ++i) {
    std::cout << arr[i] << " ";
    }    std::cout <<
std::endl;    return 0;
}
```

}

Output:

```

Output
Enter the number of elements: 5
Enter 5 integers: 66
4
7
45
5
Sorted array: 4 5 7 45 66
    
```

Time Complexity

Time Complexity:-

mergesort(arr, low, mid) $\rightarrow n/2$
 mergesort(arr, mid+1, high) $\rightarrow n/2$
 merge(arr, low, mid, high) $\rightarrow n$
 $T(n) = T(n/2) + T(n/2) + n$
 $T(n) = 2T(n/2) + n$

master method:- $T(n) = aT(n/b) + \Theta(n^c)$ $\rightarrow a=2, b=2, c=1$
 $n \log_b a = n \log_2 2 = n$
 $f(n) = n$
 $n \log_b a = n$ case: $p > -1$
 $T(n) = O(n \log^p n)$
 $= O(n \log n)$

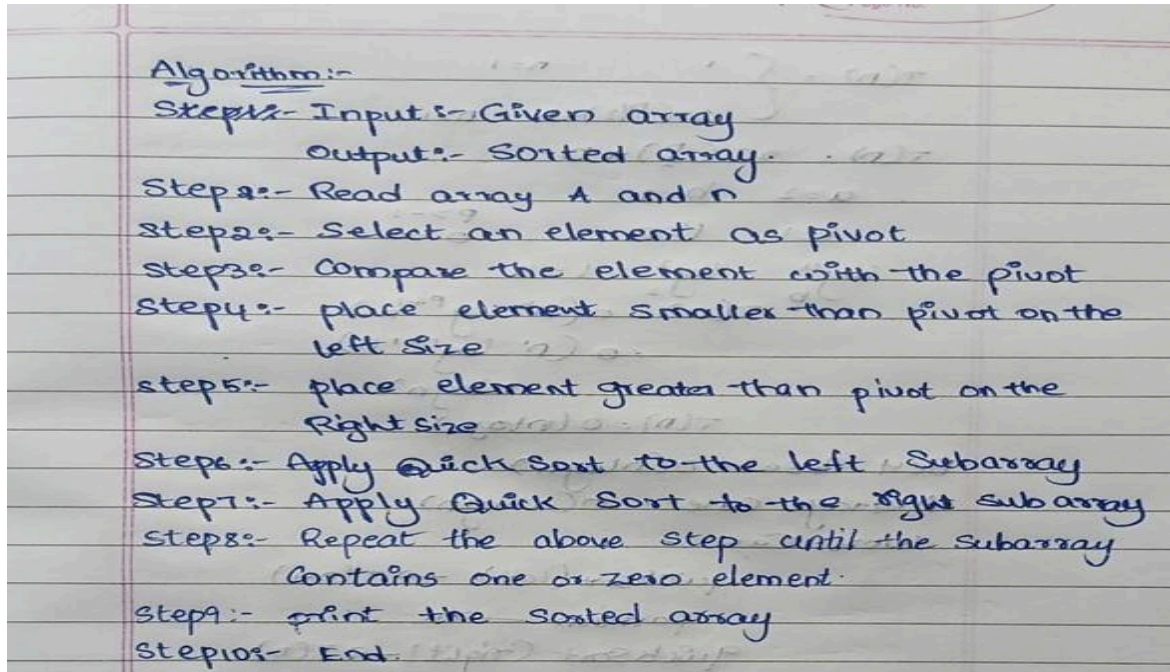
Best Case:- divides array into two halves
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 \rightarrow \dots$
 $T(n) = n \times \log_2 n$
 $T(n) = O(n \log n)$

Average Case:-
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots$
 $T(n) = n \times \log_2 n$
 $T(n) = O(n \log n)$

Worst Case:- (Merge Sort continues dividing the array into halves & merging all elements)
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$
 $T(n) = n \times \log_2 n$
 $T(n) = O(n \log n)$

Quick Sort:

Algorithm:

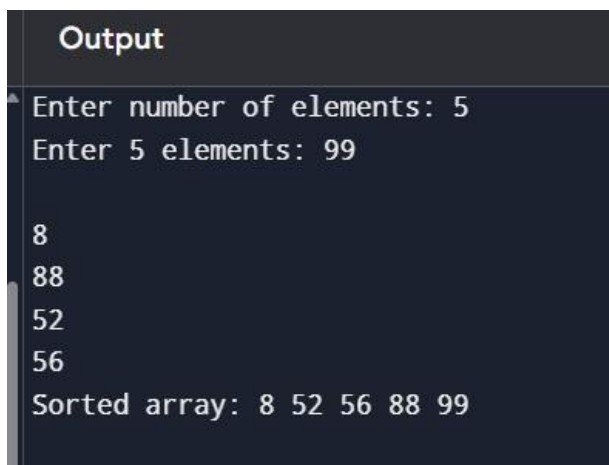


Code:

```
#include <iostream>
#include <vector>
void swap(int& a, int& b) {
    int temp = a;    a = b;    b = temp; }
int partition(std::vector<int>& arr, int low, int high){
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1],
arr[high]);    return i +
1;
} void quickSort(std::vector<int>& arr, int low, int high)
{    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
} int main(){    int n;
```

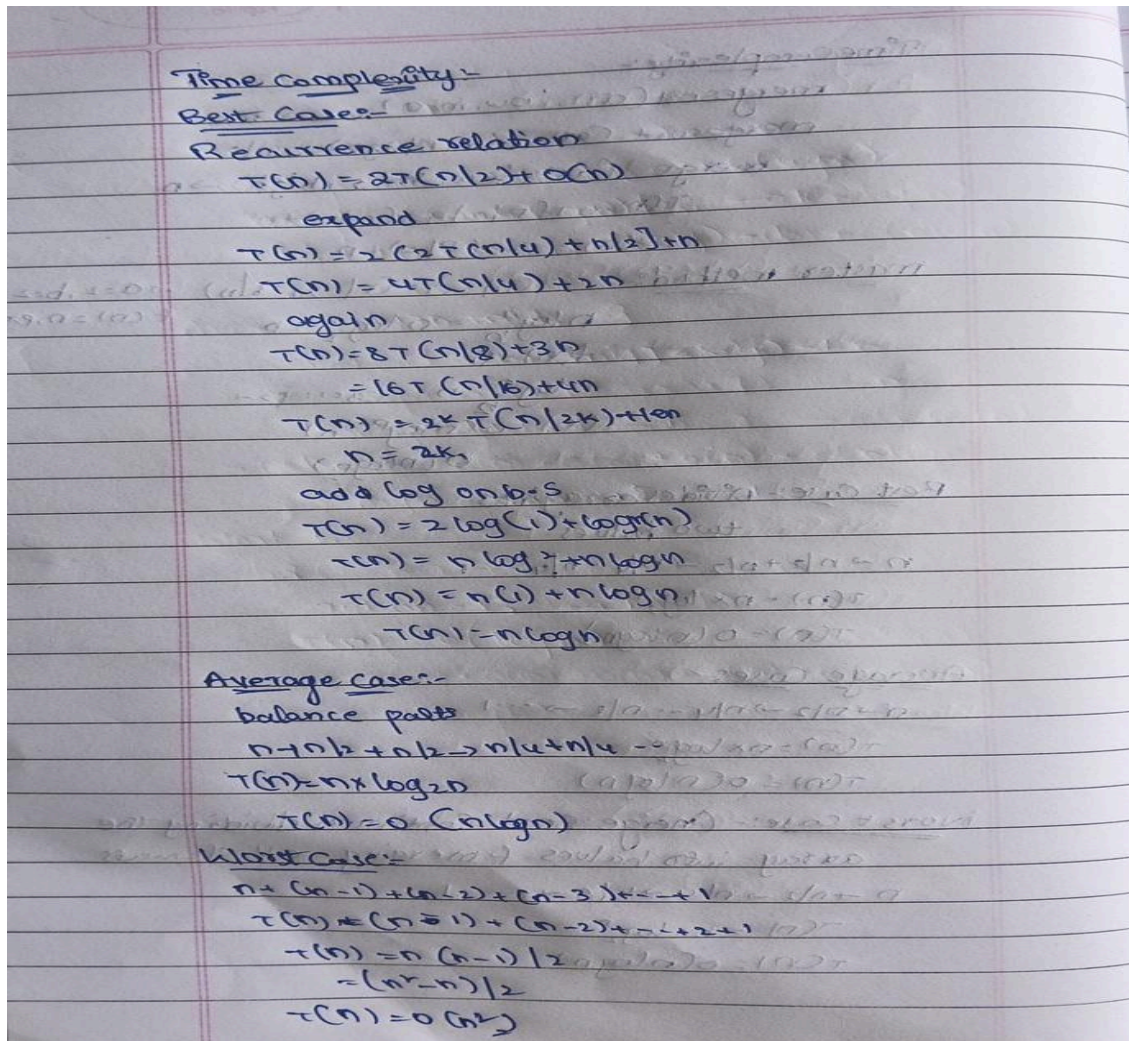
```
std::cout
<< "Enter
number of
elements:
",
std::cin >>
n;

std::vector<int> arr(n); std::cout <<
"Enter " << n << " elements: "; for (int i =
0; i < n; i++) { std::cin >> arr[i];
} quickSort(arr, 0, n - 1);
std::cout << "Sorted array: ";
for (int i = 0; i < n; i++) {
std::cout << arr[i] << " ";
} std::cout <<
std::endl; return 0;
}
```

Output:A screenshot of a terminal window with a dark background. The title bar says "Output". The text in the terminal shows the program's execution: it prompts for the number of elements (5), then prompts for 5 elements (99, 8, 88, 52, 56), and finally displays the sorted array: 8 52 56 88 99.

```
Output
Enter number of elements: 5
Enter 5 elements: 99
8
88
52
56
Sorted array: 8 52 56 88 99
```

Time complexity :



Conclusion:

The implementation and time analysis of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort helps us understand how different sorting algorithms perform for different input sizes. By analyzing their best, average, and worst cases, we can choose the most suitable algorithm for a particular problem. Merge Sort and Quick Sort are generally faster for large datasets, while Bubble, Selection, and Insertion Sort are simpler and useful for small datasets.